Group 35

Group Members: Lilitha Kwezula

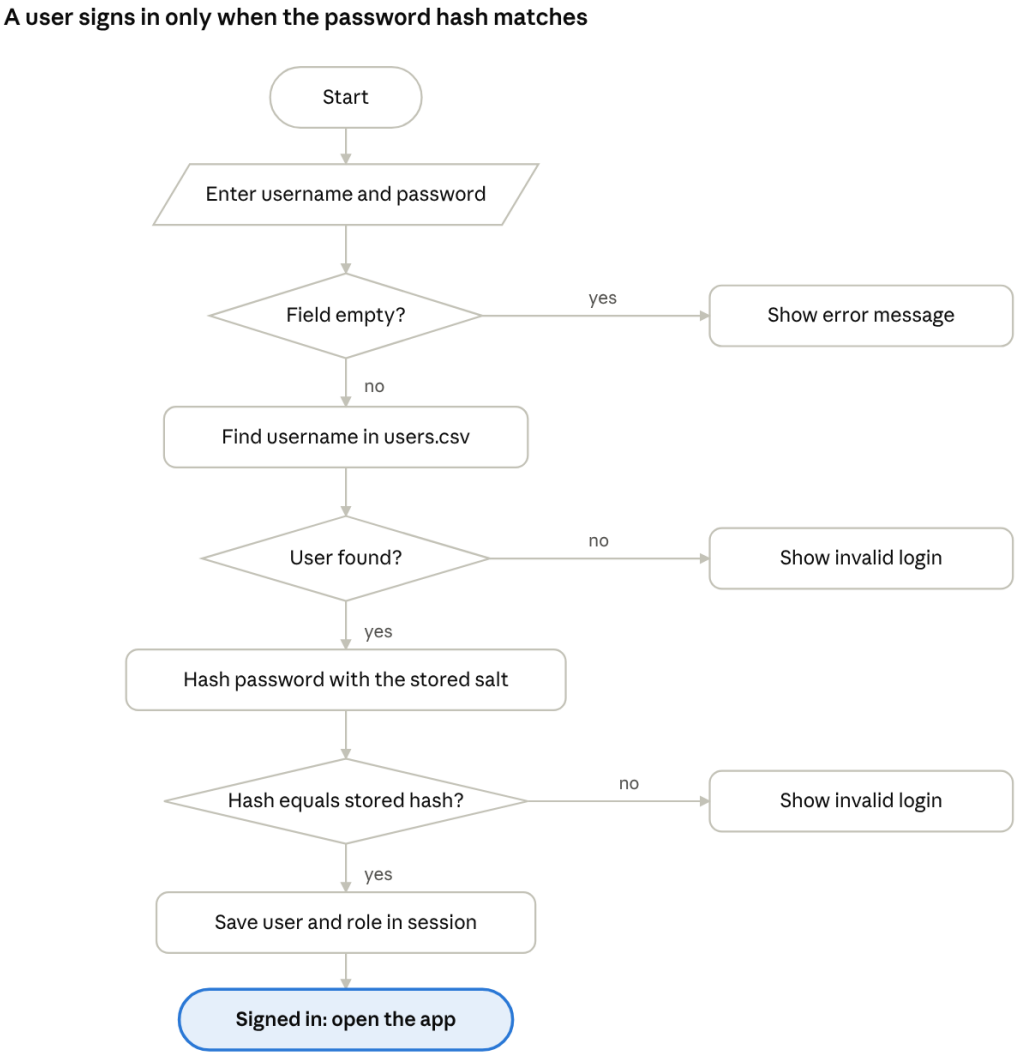
Hletelo Kubayi

Flowcharts and Pseudocode: Khwezi Mining Health, Safety and Equipment Monitoring App

1. Login (verify_user and login_page)

Purpose: authenticate a user with a username and password.

Flowchart



Pseudocode

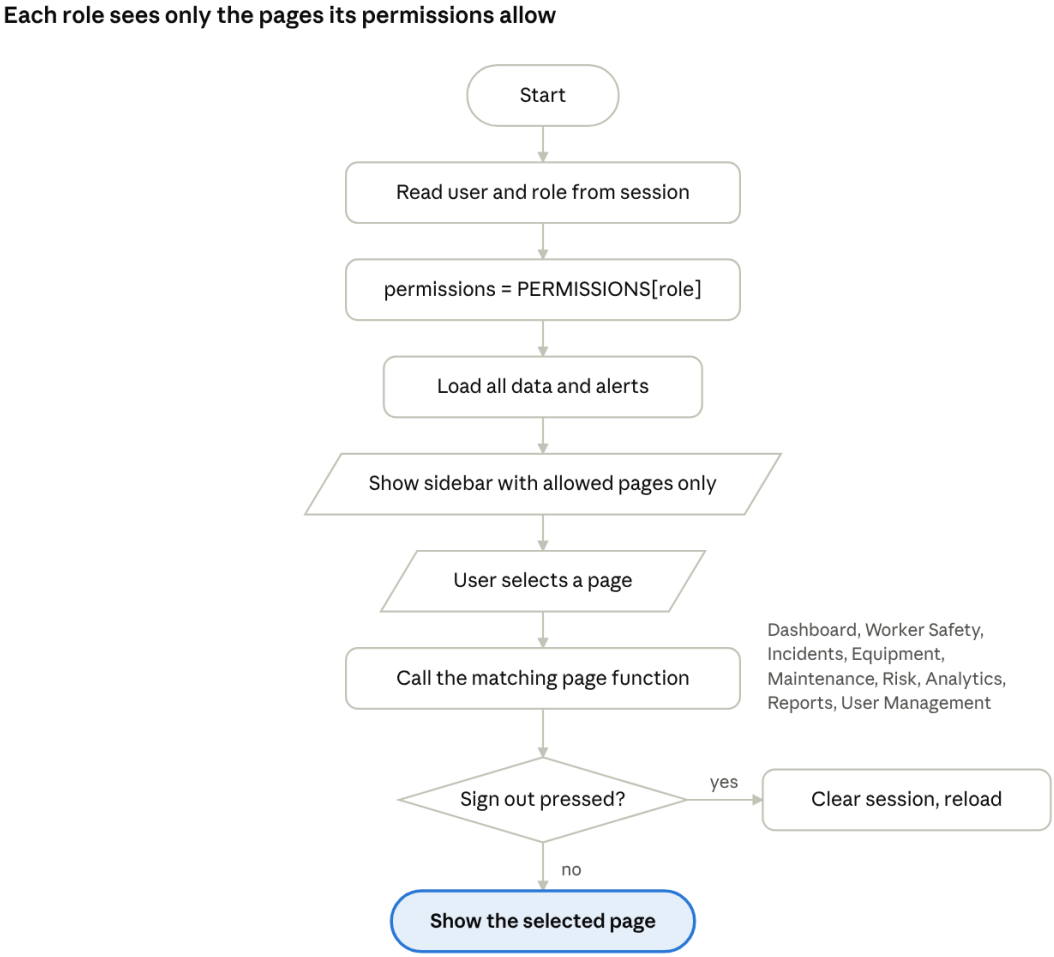
```
FUNCTION verify_user(username, password)
    username <- lowercase(trim(username))
    IF username is an alias THEN replace it with the real username
    SEARCH users.csv for a record with this username
    IF no record is found THEN
        RETURN none
    ENDIF
    hash <- PBKDF2_SHA256(password, stored salt)
    IF hash equals stored password hash THEN
        RETURN (username, name, role)
    ELSE
        RETURN none
    ENDIF
END FUNCTION

FUNCTION login_page()
    DISPLAY username field, password field and Sign in button
    WHEN Sign in is pressed
        IF username is empty OR password is empty THEN
            DISPLAY error "Enter both username and password"
        ELSE
            user <- verify_user(username, password)
            IF user exists THEN
                SAVE user in session
                SET authenticated <- TRUE
                RELOAD application
            ELSE
                DISPLAY error "Invalid username or password"
            ENDIF
        ENDIF
END FUNCTION
```

2. Navigation and role-based access control (app)

Purpose: show each user only the pages their role is allowed to use

Flowchart



navigation flow · role-based page list

The sidebar is built from the role's permissions, so a page outside them is never offered.

Pseudocode

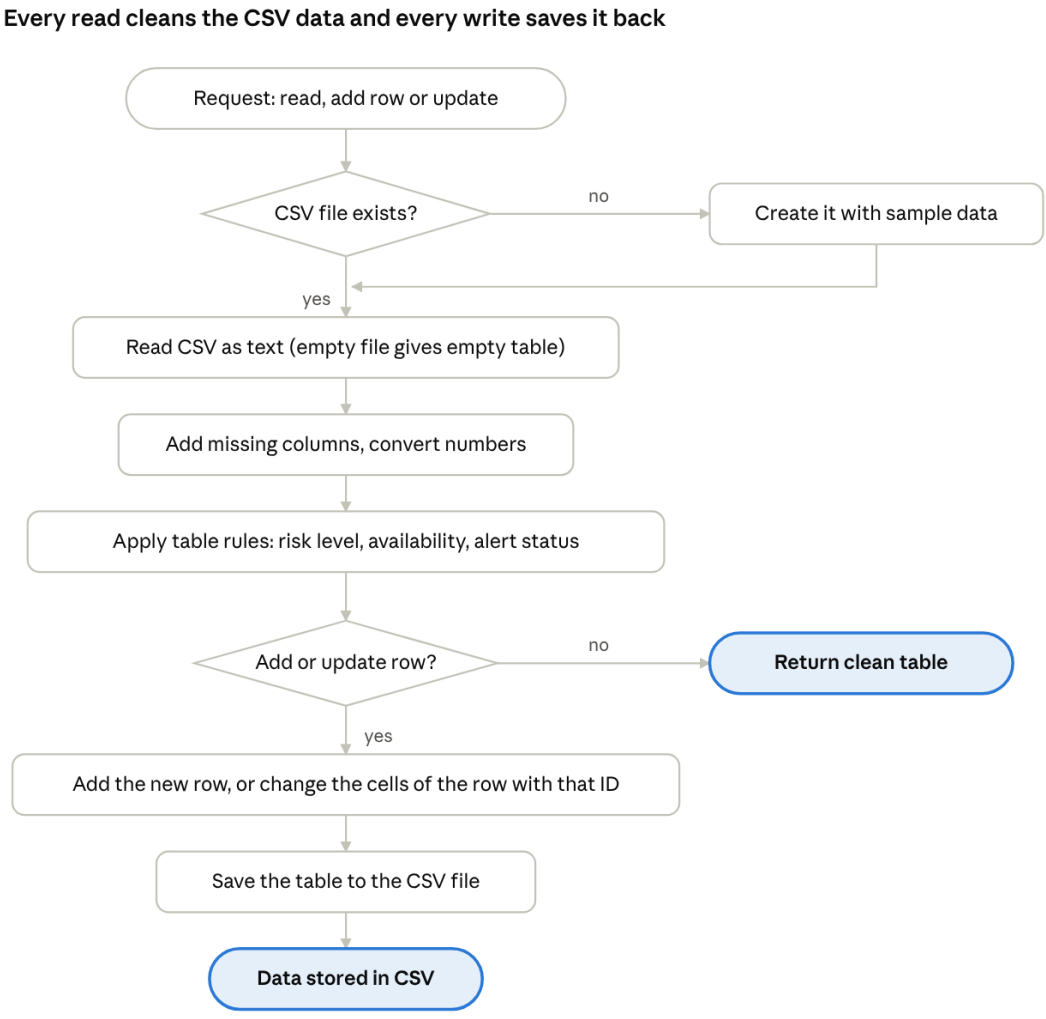
```
FUNCTION app()
    user <- user stored in session
    permissions <- PERMISSIONS[user.role]
    data <- load_all()
    DISPLAY sidebar with user name and role
    allowed_pages <- every page that is in permissions
    selected <- page chosen from allowed_pages
    DISPLAY stored feedback messages, then clear them
    IF selected = "Dashboard" THEN CALL dashboard(data, permissions)
    ELSE IF selected = "Worker Safety" THEN CALL worker_page(data)
    ELSE IF selected = "Incidents" THEN CALL incidents_page(data)
    ELSE IF selected = "Equipment" THEN CALL equipment_page(data)
    ELSE IF selected = "Maintenance" THEN CALL maintenance_page(data)
    ELSE IF selected = "Risk Assessment" THEN CALL risk_page(data)
```

```
    ELSE IF selected = "Analytics" THEN CALL analytics_page(data, permissions)
    ELSE IF selected = "Reports" THEN CALL reports_page(data, permissions, user)
    ELSE IF selected = "User Management" THEN CALL users_page()
    ENDIF
    IF Sign out is pressed THEN
        CLEAR session user and authenticated flag
        RELOAD application
    ENDIF
END FUNCTION
```

3. Data storage (load, clean, append_row, update_cells)

Purpose: read and save all records in CSV files so data persists between sessions.

Flowchart



storage flow · read, add and update

All pages read through the same clean step and write through the same save step, so rules are applied once.

Pseudocode

```
FUNCTION load(table)
    IF the CSV file does not exist THEN
        CREATE it with sample data
    ENDIF
    TRY read the CSV file
        IF file is empty THEN RETURN empty table with the correct columns
    ON ERROR
        DISPLAY error message and STOP
    RETURN table
END FUNCTION

FUNCTION get(table)
    data <- load(table)
    ADD any missing columns
    CONVERT numeric columns to numbers, others to trimmed text
    APPLY table-specific rules (risk level, availability, alert status)
    RETURN data
END FUNCTION

FUNCTION append_row(table, record)
    data <- get(table)
    ADD record as a new last row of data
    SAVE data to the CSV file
END FUNCTION

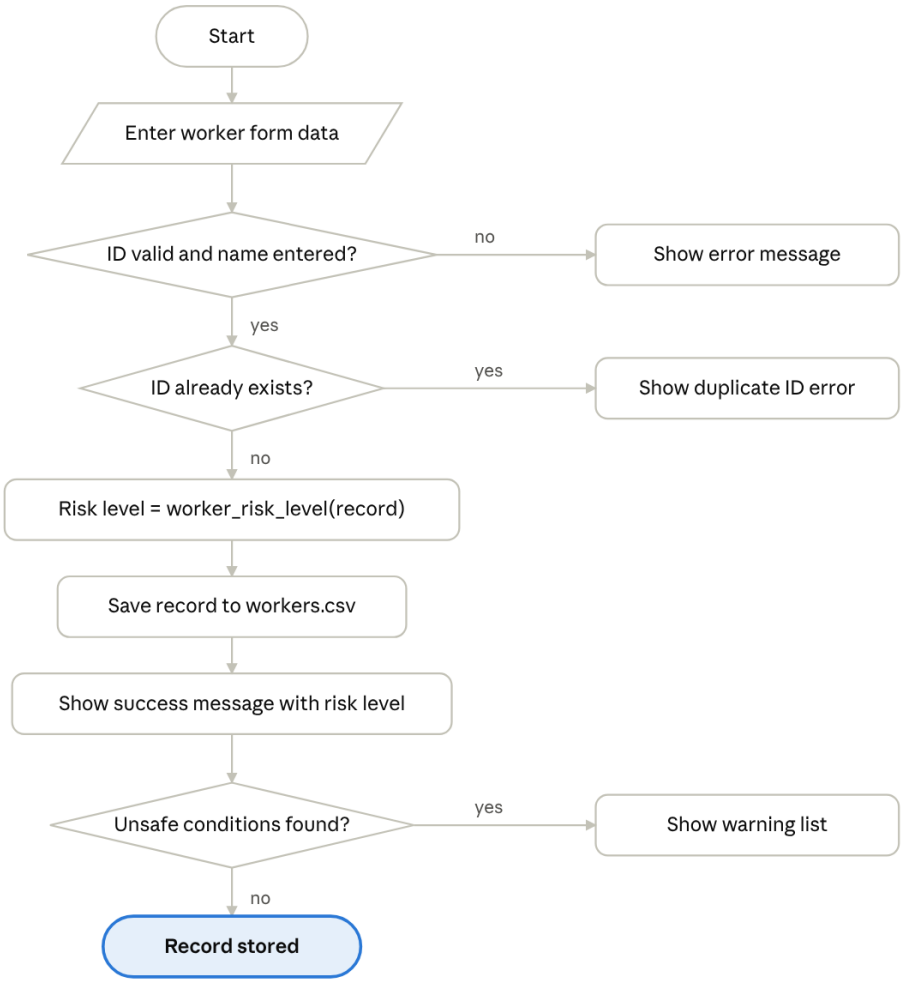
FUNCTION update_cells(table, id_column, id_value, changes)
    data <- get(table)
    FIND the row where id_column = id_value
    IF no row is found THEN RETURN FALSE
    FOR EACH (column, value) in changes
        SET that cell of the row to value
    END FOR
    SAVE data to the CSV file
    RETURN TRUE
END FUNCTION
```

4. Worker health and safety (worker_risk_level, worker_warnings, add worker)

Purpose: record worker safety information, rate the worker's risk and warn about unsafe conditions.

Flowchart

A worker record is saved only when valid, then rated and checked



Pseudocode

```
FUNCTION worker_risk_level(worker)
    points <- 0
    IF PPE < 60 THEN points <- points + 3
    ELSE IF PPE < 80 THEN points <- points + 2
    IF fatigue >= 5 THEN points <- points + 3
    ELSE IF fatigue >= 4 THEN points <- points + 2
    IF training is not "Complete" THEN points <- points + 1
    IF near_misses >= 3 THEN points <- points + 1
    IF previous_incidents >= 2 THEN points <- points + 1
    IF points <= 1 THEN RETURN "Low"
    ELSE IF points <= 3 THEN RETURN "Medium"
    ELSE IF points <= 5 THEN RETURN "High"
    ELSE RETURN "Critical"
END FUNCTION
```

```
FUNCTION worker_warnings(worker)
    warnings <- empty list
    IF PPE < 80 THEN ADD "Low PPE compliance" to warnings
    IF fatigue >= 4 THEN ADD "High fatigue level" to warnings
    IF training is not "Complete" THEN ADD "Training not complete" to warnings
    IF near_misses >= 3 THEN ADD "Repeated near misses" to warnings
    IF risk level is High or Critical THEN ADD "High worker risk" to warnings
    RETURN warnings
END FUNCTION

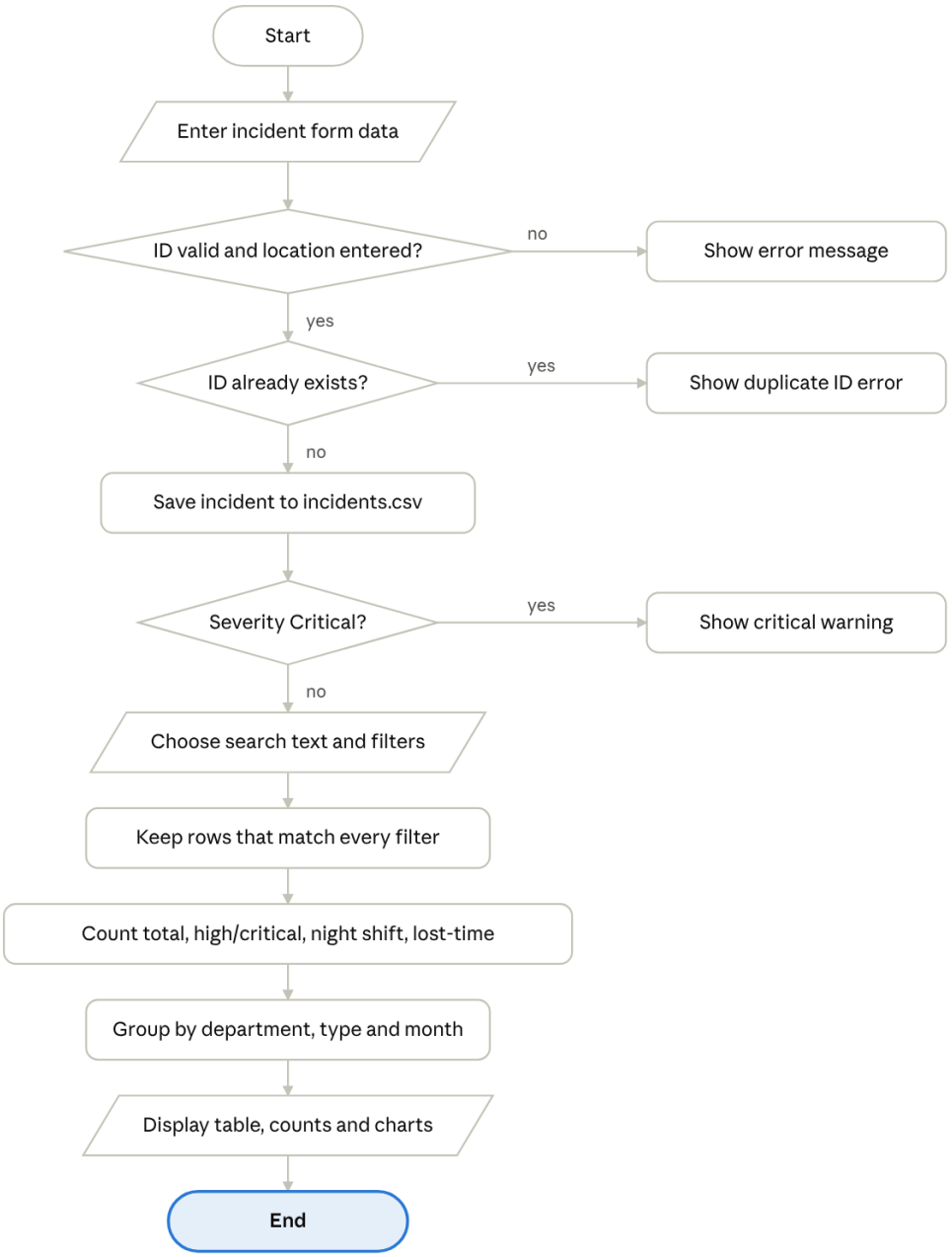
FUNCTION add_worker(form_data)
    IF worker ID is invalid OR name is empty THEN
        DISPLAY error; STOP
    ENDIF
    IF worker ID already exists THEN
        DISPLAY error "That Worker ID already exists"; STOP
    ENDIF
    record <- form_data
    record.risk_level <- worker_risk_level(record)
    append_row("workers", record)
    DISPLAY success message with the risk level
    warnings <- worker_warnings(record)
    IF warnings is not empty THEN
        DISPLAY warning listing the unsafe conditions
    ENDIF
END FUNCTION
```

5. Safety incident database (add, search, filter, count, trend)

Purpose: keep a central register of incidents that can be searched, filtered and analysed.

Flowchart

Incidents are checked on entry, then filtered, counted and trended



incident flow · add, filter, count, trend

Adding an incident and analysing the register are two stages of the same page; a critical incident also triggers a warning.

Pseudocode

```
FUNCTION add_incident(form_data)
    IF incident ID is invalid OR location is empty THEN
        DISPLAY error; STOP
    ENDIF
    IF incident ID already exists THEN
        DISPLAY error; STOP
    ENDIF
    append_row("incidents", form_data)
    DISPLAY success message
    IF severity = "Critical" THEN
        DISPLAY critical incident warning
    ENDIF
END FUNCTION

FUNCTION filter_incidents(incidents, search_text, severities, departments, shifts,
statuses, period)
    view <- incidents
    KEEP rows where severity is in severities
    KEEP rows where department is in departments
    KEEP rows where shift is in shifts
    KEEP rows where status is in statuses
    KEEP rows where date is within period
    IF search_text is not empty THEN
        KEEP rows where any column contains search_text
    ENDIF
    RETURN view
END FUNCTION

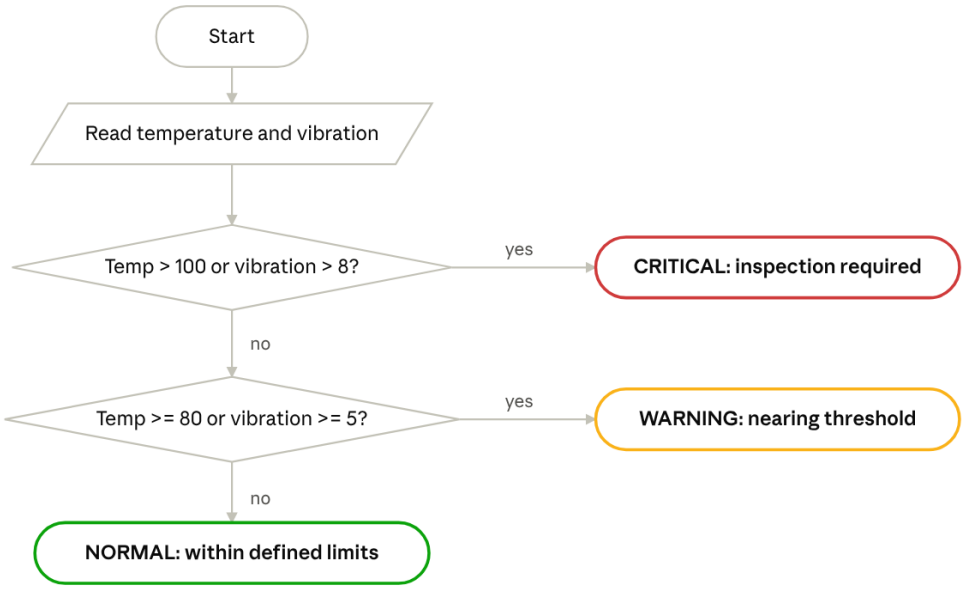
FUNCTION incidents_page(data)
    view <- filter_incidents(...)
    total <- number of rows in view
    high_critical <- count rows with severity High or Critical
    night <- count rows where shift = "Night"
    lost_time <- count rows where lost-time injury = "Yes"
    DISPLAY the counts and the table of view
    GROUP view by department and by incident type; DISPLAY bar charts
    GROUP view by month; DISPLAY trend line chart
END FUNCTION
```

6. Equipment condition monitoring (equipment_status and equipment_message)

Purpose: classify equipment as NORMAL, WARNING or CRITICAL from temperature and vibration.

Flowchart

Critical is tested first, then warning; everything else is normal



Pseudocode

```
FUNCTION equipment_status(temperature, vibration)
    IF temperature > 100 OR vibration > 8 THEN
        RETURN "CRITICAL"
    ELSE IF temperature >= 80 OR vibration >= 5 THEN
        RETURN "WARNING"
    ELSE
        RETURN "NORMAL"
    ENDIF
END FUNCTION

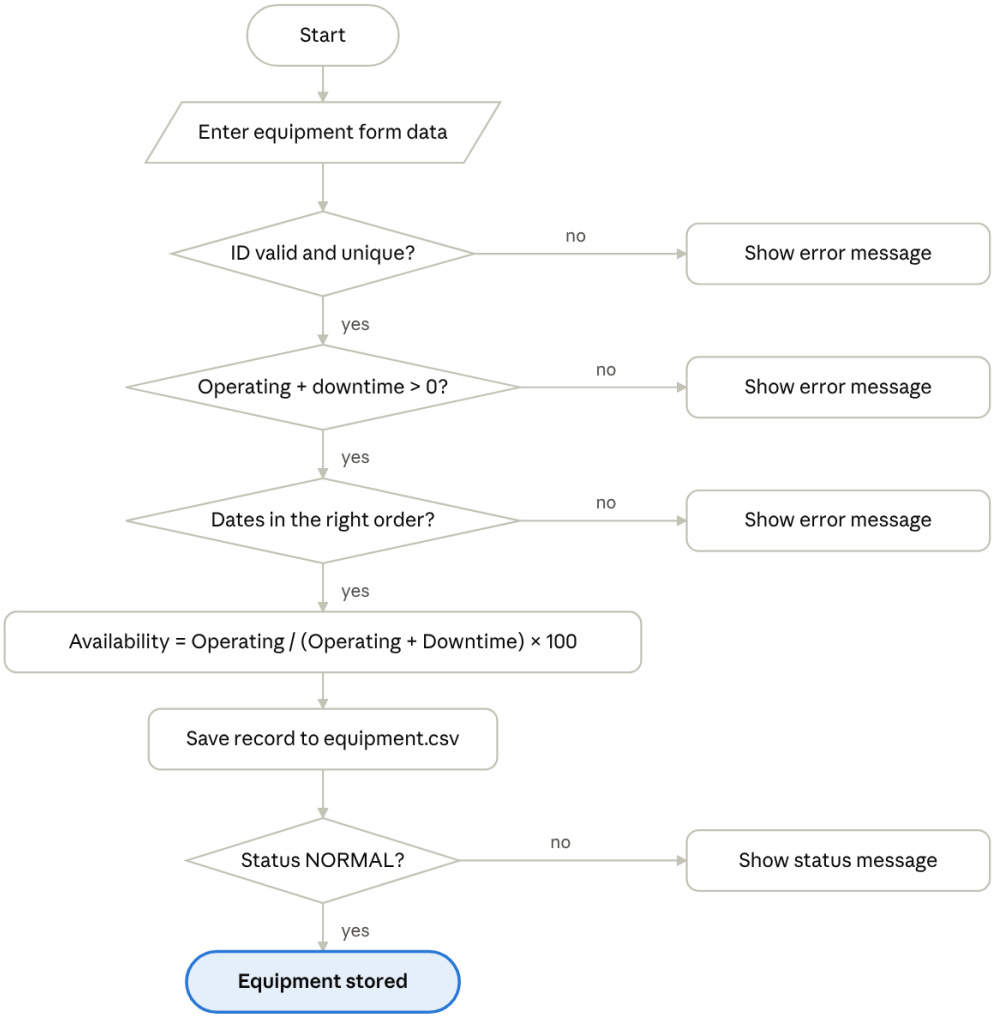
FUNCTION equipment_message(temperature, vibration)
    IF vibration > 8 THEN
        RETURN "CRITICAL: Equipment vibration exceeds predefined threshold.
Inspection required."
    ELSE IF temperature > 100 THEN
        RETURN "CRITICAL: Equipment temperature exceeds predefined threshold.
Inspection required."
    ELSE IF temperature >= 80 THEN
        RETURN "WARNING: Equipment temperature approaching predefined threshold."
    ELSE IF vibration >= 5 THEN
        RETURN "WARNING: Equipment vibration approaching predefined threshold."
    ELSE
        RETURN "NORMAL: Equipment operating within defined monitoring limits."
    ENDIF
END FUNCTION
```

7. Equipment database: add equipment and availability

Purpose: register equipment and calculate availability from operating time and downtime.

Flowchart

Availability is calculated from operating time and downtime before saving



add equipment flow · 3 checks, availability formula

Three checks guard the save; availability is calculated, never typed in.

Pseudocode

```
FUNCTION availability(operating_time, downtime)
    total <- operating_time + downtime
    IF total > 0 THEN
        RETURN (operating_time / total) * 100
    ELSE
        RETURN 0
```

```
    ENDIF
END FUNCTION

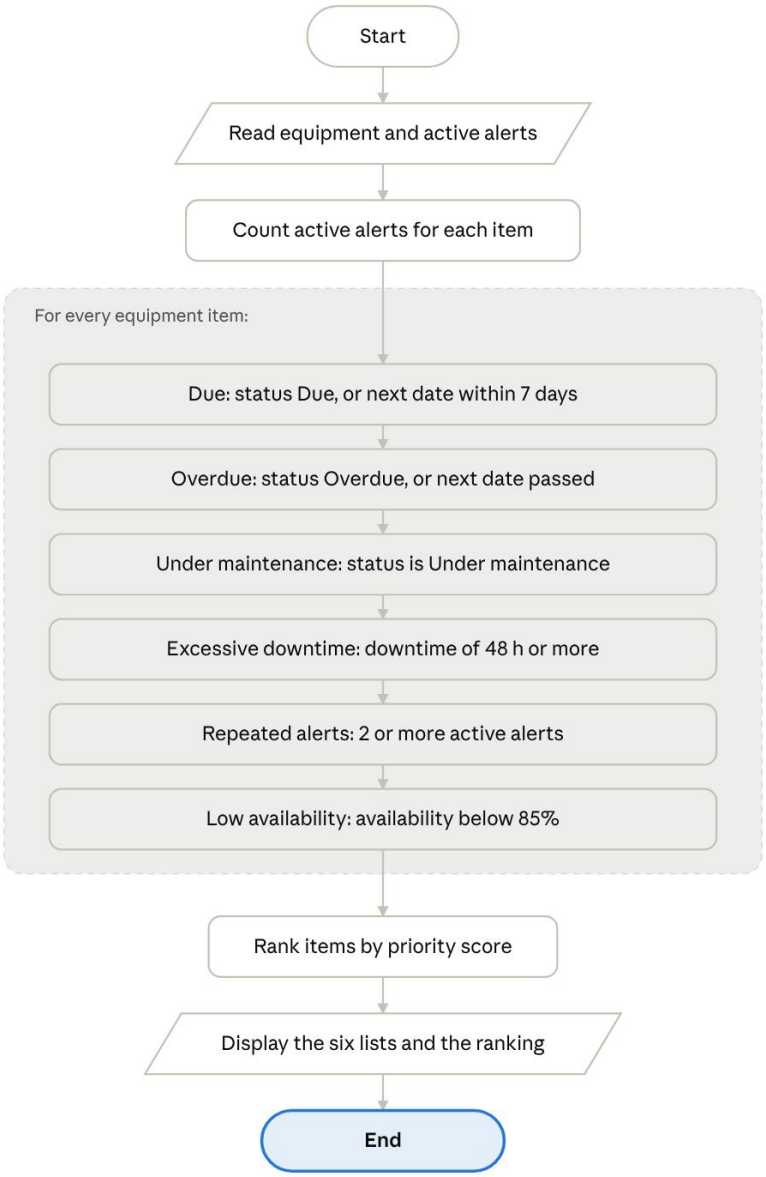
FUNCTION add_equipment(form_data)
    IF equipment ID is invalid THEN DISPLAY error; STOP
    IF equipment ID already exists THEN DISPLAY error; STOP
    IF operating_time + downtime <= 0 THEN DISPLAY error; STOP
    IF next maintenance date < last maintenance date THEN DISPLAY error; STOP
    record <- form_data
    record.availability <- availability(operating_time, downtime)
    append_row("equipment", record)
    DISPLAY success message with availability
    status <- equipment_status(temperature, vibration)
    IF status is not "NORMAL" THEN
        DISPLAY equipment_message(temperature, vibration)
    ENDIF
END FUNCTION
```

8. Equipment maintenance monitoring

Purpose: identify equipment that is due, overdue, under maintenance, has excessive downtime, repeated alerts or low availability.

Flowchart

Each equipment item is tested against six maintenance rules



Pseudocode

```
FUNCTION is_overdue(equipment)
    IF status = "Under maintenance" THEN RETURN FALSE
    IF status = "Overdue" THEN RETURN TRUE
    IF next_maintenance_date < today THEN RETURN TRUE
    RETURN FALSE
END FUNCTION

FUNCTION is_due(equipment)
    IF status = "Under maintenance" OR is_overdue(equipment) THEN RETURN FALSE
```

```
    IF status = "Due" THEN RETURN TRUE
    IF next_maintenance_date <= today + 7 days THEN RETURN TRUE
    RETURN FALSE
END FUNCTION

FUNCTION maintenance_sets(equipment, alerts)
    FOR EACH equipment item
        alert_count <- number of active alerts for the item
    END FOR
    due <- items where is_due is TRUE
    overdue <- items where is_overdue is TRUE
    under_maintenance <- items where status = "Under maintenance"
    high_downtime <- items where downtime >= 48
    repeated_alerts <- items where alert_count >= 2
    low_availability <- items where availability < 85
    RETURN all six lists
END FUNCTION

FUNCTION priority_score(equipment)
    score <- 0
    IF alert status = CRITICAL THEN score <- score + 4
    ELSE IF alert status = WARNING THEN score <- score + 2
    IF is_overdue THEN score <- score + 3
    ELSE IF is_due THEN score <- score + 1
    IF availability < 85 THEN score <- score + 1
    IF downtime >= 48 THEN score <- score + 1
    score <- score + MIN(alert_count, 3)
    RETURN score
END FUNCTION
```

9. Risk assessment (classify_risk and save assessment)

Purpose: calculate Risk Score = Likelihood x Consequence and classify the risk.

Pseudocode

```
FUNCTION classify_risk(score)
    IF score <= 4 THEN RETURN "Low"
    ELSE IF score <= 9 THEN RETURN "Medium"
    ELSE IF score <= 16 THEN RETURN "High"
    ELSE RETURN "Critical"
    ENDIF
END FUNCTION

FUNCTION save_risk_assessment(form_data)
    IF assessment ID is invalid OR hazard is empty THEN DISPLAY error; STOP
    IF assessment ID already exists THEN DISPLAY error; STOP
    IF review date < assessment date THEN DISPLAY error; STOP
    score <- likelihood * consequence
    level <- classify_risk(score)
    append_row("risk", form_data, score, level)
    DISPLAY success message with score and level
    IF level = "High" OR level = "Critical" THEN
        DISPLAY warning to review controls with safety personnel
```

```
    ENDIF
END FUNCTION
```

10. Alerts and notifications (build_alerts)

Purpose: automatically create alerts from the predefined conditions.

Pseudocode

```
FUNCTION build_alerts(data)
    alerts <- empty list
    FOR EACH equipment item
        IF vibration > 8 THEN ADD CRITICAL alert
        ELSE IF vibration >= 5 THEN ADD WARNING alert
        IF temperature > 100 THEN ADD CRITICAL alert
        ELSE IF temperature >= 80 THEN ADD WARNING alert
        IF downtime >= 48 THEN ADD WARNING alert (excessive downtime)
        IF availability < 85 THEN ADD WARNING alert (low availability)
        IF brake, tyre or engine status = "Poor" THEN ADD WARNING alert
        IF is_overdue(item) THEN ADD WARNING alert (overdue maintenance)
    END FOR
    FOR EACH worker
        IF PPE < 80 THEN ADD WARNING alert
        IF fatigue >= 5 THEN ADD CRITICAL alert
        ELSE IF fatigue >= 4 THEN ADD WARNING alert
    END FOR
    FOR EACH incident
        IF severity = "Critical" AND status is not "Closed" THEN ADD CRITICAL alert
    END FOR
    FOR EACH risk assessment
        IF level is High or Critical AND status is not "Closed" THEN ADD alert
    END FOR
    SORT alerts so CRITICAL alerts come first
    RETURN alerts
END FUNCTION

FUNCTION visible_alerts(alerts, permissions)
    RETURN only the alerts whose category the user is permitted to see
END FUNCTION
```

11. Mining monitoring dashboard

Purpose: show key performance indicators and active alerts for the user's role.

Pseudocode

```
FUNCTION dashboard(data, permissions)
    kpis <- empty list
    IF "Worker Safety" in permissions THEN
        ADD total workers, near misses, average PPE compliance
    ENDIF
    IF "Incidents" in permissions THEN
        ADD total incidents, open incidents
    ENDIF
    IF "Risk Assessment" in permissions THEN
```

```
        ADD open high-risk observations
    ENDIF
    IF "Equipment" in permissions THEN
        ADD total equipment, equipment available, under maintenance,
            equipment alerts, average availability, critical alerts
    ENDIF
    DISPLAY kpis
    alerts <- visible_alerts(data.alerts, permissions)
    IF alerts is empty THEN DISPLAY "No active alerts"
    ELSE DISPLAY alert table and alert notices
    ENDIF
    DISPLAY charts the user is permitted to see
END FUNCTION
```

12. Interactive data analysis (analytics_page)

Purpose: answer mining engineering questions from Appendix A using the stored data.

Pseudocode

```
FUNCTION analytics_page(data, permissions)
    available <- questions whose required permissions are all in permissions
    category <- category chosen by the user
    question <- question chosen from that category
    CALL the function that answers the question, for example:
        GROUP records by the relevant field
        COUNT, SUM or AVERAGE the values
        SORT results from highest to lowest
        DISPLAY the answer as a number, table and chart
END FUNCTION
```

13. Report generation (build_report)

Purpose: create a downloadable monitoring summary containing only the sections a role may view.

Pseudocode

```
FUNCTION build_report(data, permissions, user)
    report <- header with date, user name and role
    ADD section "Active alerts" (visible alerts only)
    IF "Worker Safety" in permissions THEN ADD worker safety summary
    IF "Incidents" in permissions THEN ADD incident summary
    IF "Equipment" in permissions THEN
        ADD equipment condition summary
        ADD maintenance status summary
    ENDIF
    IF "Risk Assessment" in permissions THEN ADD risk summary
    ADD section "Recommendations" from generated recommendations
    RETURN report
END FUNCTION

FUNCTION reports_page(data, permissions, user)
    report <- build_report(data, permissions, user)
    DISPLAY report preview
    OFFER report as a text file download
```

```
    FOR EACH dataset the user is permitted to view
        OFFER the dataset as a CSV download
    END FOR
END FUNCTION
```

14. User management (Administrator only)

Purpose: create users, change roles, reset passwords and delete users.

Pseudocode

```
FUNCTION add_user(username, name, role, password, confirm)
    IF username is invalid OR name is empty THEN DISPLAY error; STOP
    IF username already exists THEN DISPLAY error; STOP
    IF password length < 8 OR password <> confirm THEN DISPLAY error; STOP
    salt <- random value
    hash <- PBKDF2_SHA256(password, salt)
    append_row("users", username, name, role, salt, hash)
    DISPLAY success message
END FUNCTION

FUNCTION change_user(username, new_role, new_password)
    IF user is the last Administrator AND new_role is not Administrator THEN
        DISPLAY error; STOP
    ENDIF
    IF new_password is given AND length < 8 THEN DISPLAY error; STOP
    update_cells("users", username, role and, if given, new salt and hash)
    DISPLAY success message
END FUNCTION

FUNCTION delete_user(username, confirmed)
    IF NOT confirmed THEN DISPLAY error; STOP
    IF username = signed-in user THEN DISPLAY error; STOP
    IF user is the last Administrator THEN DISPLAY error; STOP
    REMOVE the user from users.csv
    DISPLAY success message
END FUNCTION
```